MyPredictive_Edge — Complete Project Roadmap

Start-to-Finish roadmap for building the Predictive Edge backend step by step.

Core business idea: External signals → anomaly/shortage alert → optimizer → routing decision → human approval → inventory transfer.

1. Working Rules for This Project

Rule	How we will work
Understand first	Before coding, explain what the module does, why it exists, and how it connects to the business flow.
One step at a time	Do not implement many modules together. Finish and test the current step before moving forward.
Reference architecture	Use the supplied reference project as the architectural guide, but adapt code to MyPredictive_Edge.
No blind copy-paste	Never copy reference code if its tables, fields, authentication, or assumptions do not match our project.
Test every module	After implementation, run the API/database test and verify the result before continuing.
Security	Use authentication, RBAC, organization isolation, validation, logging, and safe error handling.
Human-in-the-loop	Optimization creates a pending decision; inventory movement happens only after approval/override.

2. Current Status — Already Completed

✓ FastAPI project foundation

✓ Configuration and .env loading

✓ Supabase sync/async client

✓ SQLAlchemy database core

✓ Logging with request/correlation context

✓ Security headers and request middleware

✓ Centralized application exceptions

✓ SlowAPI rate limiting

✓ Supabase authentication: register, login, me, logout, refresh

✓ Authentication testing

✓ Locations database table + CRUD + validation

3. Phase-by-Phase Roadmap

Phase 1 — Foundation — *STATUS: DONE*

FastAPI app, configuration, environment variables, Supabase, database core, middleware, logging, exceptions, rate limiting, authentication, and locations CRUD.

Target: Result: a secure and testable backend foundation.

Phase 2 — Organization Management — *STATUS: NEXT*

Create organizations and establish the organization ownership boundary. Add organization model/schema/service/routes and connect users to organizations.

Target: Result: every business resource can eventually belong to the correct organization.

Phase 3 — Roles & RBAC — *STATUS: PENDING*

Implement organization roles and permissions. Core tables: organization_roles and organization_role_permissions. Connect users to role + organization. Add permission dependencies such as require_permission(...). Add RBAC caching where useful.

Target: Example permissions: signals.ingest, alerts.read, optimize, routing.read, routing.approve, routing.reject, routing.override.

Phase 4 — Invitations — *STATUS: PENDING*

Implement organization invitations, token handling, expiry, acceptance, role assignment, and invitation email. Integrate the existing SendGrid email service safely.

Target: Result: admins/managers can invite users into an organization.

Phase 5 — Audit Logging — *STATUS: PENDING*

Use the existing audit service to record important actions such as invitation, role changes, optimization, approval, rejection, override, and inventory movement.

Target: Audit failure must not break the primary business operation.

Phase 6 — Inventory — *STATUS: PENDING*

Create inventory as the core supply-chain data layer. Suggested fields: id, location_id, sku, stock_level, safety_stock_level, timestamps. Add constraints and CRUD/services as needed.

Target: Example: Delhi umbrella stock = 1000; Mumbai = 100; Bengaluru = 150.

Phase 7 — Signals & Signal Ingestion — *STATUS: PENDING*

Build the signals module and ingestion pipeline. Use adapters for weather, events, holidays, news, and trends. Run independent external adapters concurrently with asyncio.gather and safely handle adapter failures.

Target: Endpoint concept: POST /api/v1/signals/ingest.

Phase 8 — Anomaly Alerts — *STATUS: PENDING*

Create anomaly_alerts schema/model/service. Convert meaningful signals into business alerts. Support active, acknowledged, and resolved states as required.

Target: Example: Mumbai umbrella demand risk becomes a high-severity shortage alert.

Phase 9 — Warehouse Tools — *STATUS: PENDING*

Create controlled tools for the optimizer: get_open_alerts, get_inventory, get_locations, and create_routing_decision. The LLM must not receive arbitrary SQL/database access.

Target: Result: the AI can inspect only controlled business operations/data.

Phase 10 — LLM Reasoning — *STATUS: PENDING*

Implement MILPProblemSpec with Pydantic. Connect Groq/LLM tool calling. Give the model the alert and controlled warehouse tools. Validate the final JSON output with Pydantic.

Target: LLM proposes source, destination, SKU, quantity, constraints, objective, and reasoning.

Phase 11 — Mathematical Optimizer / MILP — *STATUS: PENDING*

Implement the solver using PuLP. Hard constraints must validate source stock and destination remaining capacity. The solver, not the LLM, is responsible for mathematical feasibility.

Target: Result: OPTIMAL or INFEASIBLE with a validated transfer quantity.

Phase 12 — Optimizer Pipeline — *STATUS: PENDING*

Connect alert → LLM → warehouse tools → inventory/capacity checks → MILP solver → routing decision. Add retry/self-correction when a proposed plan is infeasible.

Target: After successful optimization, create a PENDING routing decision.

Phase 13 — Routing Decisions — *STATUS: PENDING*

Create routing_decisions schema/model/service/routes. Support listing decisions and filtering by status. Store source, destination, SKU, quantity, solver status, notes, cost/optimization metadata, and execution state.

Target: Endpoint concepts: GET /decisions and POST /optimize.

Phase 14 — Human Approval Workflow — *STATUS: PENDING*

Implement approve, reject, and override actions with RBAC. Only approved/overridden decisions may execute inventory movement. Prevent invalid state transitions.

Target: PENDING → APPROVED or PENDING → REJECTED. Override represents an authorized human intervention.

Phase 15 — Inventory Fulfillment / Transfer — *STATUS: PENDING*

Execute source decrement + destination increment after approval. Validate sufficient stock and destination capacity. Design this carefully for consistency/atomicity.

Target: Example: Delhi 1000 → 700 and Mumbai 100 → 400 after a 300-unit approved transfer.

Phase 16 — Alert Resolution — *STATUS: PENDING*

Allow authorized users/processes to resolve or acknowledge alerts after the relevant business action. Connect alert lifecycle with optimization status where appropriate.

Target: Result: alerts do not remain active forever after the problem is handled.

Phase 17 — End-to-End Testing — *STATUS: PENDING*

Test authentication, RBAC, organization isolation, locations, inventory, signal ingestion, alert creation, optimizer feasibility, routing decisions, approval/rejection/override, and inventory transfer.

Target: Include success cases, validation errors, unauthorized access, missing data, insufficient stock, insufficient capacity, and external API failures.

Phase 18 — Production Hardening — *STATUS: PENDING*

Improve secrets management, structured logging, rate limits, timeouts, retries, error handling, database consistency, idempotency, monitoring, and API documentation.

Target: Goal: make the project production-oriented rather than only demo-ready.

4. Final Business Flow

```
External Signals
↓
Signal Ingestion
↓
Anomaly Detection / Business Rules
↓
anomaly_alerts
↓
Optimizer
• LLM Reasoning
• Warehouse Tools
• MILP / PuLP Solver
↓
routing_decisions (PENDING)
↓
Human
• APPROVE
• REJECT
• OVERRIDE
↓
Inventory Fulfillment
↓
Source stock decreases + Destination stock increases
↓
Alert / Decision lifecycle updated
```

5. Final Target Architecture

```
Client
↓
FastAPI
↓
Middleware / Logging / Rate Limit
↓
Authentication
↓
Organization Context + RBAC
↓
Router
↓
Pydantic Validation
↓
Service Layer
↓
Supabase / PostgreSQL
↓
Business Modules:
Organizations | Users | Roles | Invitations
Locations | Inventory
Signals | Alerts
Optimizer | Routing
Audit | Fulfillment
```

6. Important Reference Services We Will Reuse/Adapt

Service	Purpose
signal_ingestion_service.py	Fetch and normalize external signals and generate alert candidates.
warehouse_tools.py	Controlled data access for the optimizer/LLM.
llm_service.py	LLM reasoning + structured MILPProblemSpec generation.
solver_service.py	Hard mathematical feasibility and quantity optimization using PuLP.

optimizer_pipeline.py	Orchestrates alert → LLM → tools → solver → routing decision.
RBAC service	Loads user organization/role/permissions and supports permission checks.
invite email service	Sends organization invitation emails through SendGrid.
audit service	Writes business-action audit records without breaking primary operations.

7. Database Target Relationships

```
organizations
• users
• role_id -> organization_roles
• organization_role_permissions
• invitations
• locations
• inventory
• anomaly_alerts
↓
routing_decisions
↓
inventory transfer / fulfillment

audit_log records important actions across the system.
```

8. Development Sequence We Will Follow

1. Organization
2. RBAC / permissions
3. Invitations + audit
4. Inventory
5. Signals
6. Anomaly Alerts
7. Warehouse Tools
8. LLM service
9. MILP solver
10. Optimizer pipeline
11. Routing decisions
12. Approval / Reject / Override
13. Inventory transfer
14. Alert resolution
15. End-to-end tests
16. Production hardening

Current next step: Phase 2 — Organization Management.